M2S-99: Best Practice Summary

> **Series:** M2S | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/30/2026

A definitive, actionable reference of every best practice extracted from the M2S Managed-to-SaaS Migration series (notebooks 01–09). Each practice specifies the exact setting or action, its priority, and its step. No hedging—follow these and your migration succeeds.

The 9-Step Migration Framework

The M2S series follows a structured, step-based approach to Managed-to-SaaS migration:

Step	Name	Focus
1	**Discover**	Understand SaaS Differences
2	**Strategize**	Define Your Migration Approach
3	**Design**	Create Target Architecture
4	**Prepare**	Readiness and Pre-Migration
5	**Execute**	Migrate Configuration and Agents
6	**Integrate**	Reconnect Integrations
7	**Expand**	Adopt New SaaS Capabilities
8	**Enable**	User Enablement and Communication
9	**Optimize**	Validate, Optimize, and Decommission

> **OneAgent Attribute Enrichment (1.331+):** OneAgent can enrich all telemetry (metrics, spans, logs, events) with primary fields (`dt.security_context` , `dt.cost.costcenter`) and primary tags (`primary_tags.environment` , `primary_tags.team`) at the source. More efficient than auto-tags — feeds directly into OpenPipeline routing, bucket assignment, and Grail permissions. Configure via `oneagentctl --set-host-tag` or `--set-host-tag` at install time. See docs.

The 11-Step Order of Operations (Upgrade Phase)

Within the upgrade phase, follow this precise execution order:

#	Operation	Description
1	**Assess**	Inventory current environment

#	Operation	Description
2	**Provision**	SaaS tenant and access (SSO)
3	**Install**	New ActiveGates in parallel with old ActiveGates
4	**Migrate**	Configuration and integrations
5	**Rebuild**	Dashboards, zones, alerts
6	**Redirect**	OneAgents to SaaS
7	**Reconnect**	Integrations and extensions
8	**Migrate**	Any remaining configuration and integrations
9	**Validate**	Data flow and performance
10	**Cutover**	Full switch to SaaS
11	**Decommission**	Managed environment

Table of Contents

---

1. Step 1: Discover — Understand SaaS Differences

Practice	Recommended Setting/Value	Priority
Complete entity discovery before strategy	Run Entities API v2 inventory: `type(HOST)`, `type(SERVICE)`, `type(APPLICATION)`, `type(SYNTHETIC_TEST)`. Document counts in a spreadsheet before any other planning work.	Critical
Plan for historic data loss	Historic metrics, logs, traces, problems, and session data **cannot be migrated**. Export critical reports before shutdown. Run dual environments to build SaaS baselines.	Critical

Practice	Recommended Setting/Value	Priority
Enforce host limit per tenant	Maximum **25,000 monitored hosts** per SaaS tenant. Split into multiple tenants if consolidating Managed clusters exceeds this.	Critical
Align Managed and SaaS versions	Both environments must be on the **same major version** (e.g., both 1.294.x) before using SaaS Upgrade Assistant.	Critical
Use SaaS Upgrade Assistant for most migrations	Install the app on the target SaaS tenant. Assign `upgrade-assistant:environments:write` IAM policy to migration users. The SaaS Upgrade Assistant is the recommended tool for the majority of migrations.	Recommended
Use Monaco for tenant consolidation	When consolidating multiple Managed tenants with **identical configuration names**, use Monaco—the SaaS Upgrade Assistant cannot resolve name conflicts.	Recommended

2. Step 2: Strategize — Define Your Migration Approach

Practice	Recommended Setting/Value	Priority
Choose migration approach by environment size	<500 hosts: Big Bang. 500–2,000 hosts: Phased by environment. >2,000 hosts or complex integrations: Phased by region/application.	Critical
Define measurable success criteria	Set targets: 100% host coverage, 100% service discovery, <15 min data gaps, 100% alert delivery, 100% dashboard availability, 100% integration success.	Critical
Budget time for the 10% manual items (90/10 rule)	90% of configs migrate automatically; the remaining 10% (credentials, webhooks, custom scripts) takes 90% of the manual effort. Plan accordingly.	Recommended
Engage Dynatrace Professional Services early	Involve account team and PS during the Strategize phase, not during execution. Early engagement prevents avoidable rework.	Recommended
Choose one migration tool	Pick **one** primary tool: SaaS Upgrade Assistant (recommended), Monaco, Terraform, or Settings API. Never mix approaches—Monaco YAML conflicts with the SaaS Upgrade Assistant.	Critical

3. Step 3: Design — Create Target Architecture

Practice	Recommended Setting/Value	Priority
Open outbound 443 to SaaS endpoints	Allow HTTPS (port **443**) from all monitored hosts and ActiveGates to `{tenant-id}.live.dynatrace.com` and `{tenant-id}.apps.dynatrace.com`.	Critical

Practice	Recommended Setting/Value	Priority
Use ActiveGate routing for restricted networks	If hosts cannot reach the internet directly, deploy Environment ActiveGates. OneAgents connect to AG on port **9999**; AG connects outbound on **443**.	Critical
Recreate Network Zones in SaaS before migrating	Network Zone configuration does **not** transfer automatically. Create zones in SaaS via Settings > Network zones or the API, then assign AGs and OneAgents.	Critical
Deploy minimum 2 ActiveGates per network zone	Provides high availability within each zone. OneAgent auto-discovers available AGs.	Recommended
Configure SAML SSO with full message signing	IdP must sign the **entire SAML message**, not just the assertion. Azure AD meets this requirement by default. Failure causes authentication errors.	Critical
Confirm data residency region at provisioning	Select the correct region (US, EU, APAC) during tenant provisioning. **Cannot be changed** after provisioning.	Critical
Enforce TLS 1.2+ for all traffic	Dynatrace SaaS enforces TLS 1.2 minimum. Verify no legacy TLS 1.0/1.1 configurations on proxies or load balancers.	Critical
Ensure DNS resolves SaaS domains	Verify DNS resolution for `{tenant-id}.live.dynatrace.com`, `{tenant-id}.apps.dynatrace.com`, and `*.dynatrace.com`.	Critical
Match network zones to physical topology	Name zones by datacenter or region (e.g., `datacenter-east`, `aws-us-east-1`). Define alternative zones for failover.	Recommended
Size ActiveGates by host count	Up to 500 hosts: **2 CPU / 4 GB RAM / 20 GB disk**. 500–1,500: **4 CPU / 8 GB / 40 GB**. 1,500–5,000: **8 CPU / 16 GB / 80 GB**. >5,000: scale horizontally with multiple AGs.	Recommended

4. Step 4: Prepare — Readiness and Pre-Migration

Practice	Recommended Setting/Value	Priority
Require Managed cluster v1.294+	SaaS Upgrade Assistant requires **Dynatrace Managed version 1.294 or later**. Upgrade the Managed cluster first if needed.	Critical
Coordinate dual licensing	Work with Dynatrace account team to arrange temporary overlap licensing for the period both Managed and SaaS run simultaneously.	Critical
Deploy new SaaS ActiveGates in parallel	Install new SaaS-connected AGs alongside existing Managed AGs **before** migrating OneAgents. Validates connectivity without impacting monitored hosts.	Critical
Install SaaS Upgrade Assistant	Install the app on the target SaaS tenant and verify connectivity to the Managed cluster. Confirm all required permissions are in place.	Critical
Announce configuration freeze	Notify all teams that no configuration changes should be made to the Managed environment during the migration window. Document the freeze start and end dates.	Recommended

Practice	Recommended Setting/Value	Priority
Test rollback procedure	Document the Managed server URL and token. Rollback: `oneagentctl --set-server="https://{managed-cluster}/communication"` + restart. Test on one host before bulk migration.	Critical
Test connectivity before migration	Run `curl -v https://{tenant-id}.live.dynatrace.com/api/v1/time` from monitored hosts and ActiveGate servers before scheduling the migration window.	Critical
Verify OneAgent versions are within support window	OneAgent support window is **9 months (Standard) / 12 months (Enterprise)**. Check oldest deployed versions before migration. Upgrade outdated agents first.	Recommended

5. Step 5: Execute — Migrate Configuration and Agents

Practice	Recommended Setting/Value	Priority
Migrate configurations BEFORE redirecting OneAgents	Deploy foundational settings (management zones, tags, service detection rules) to SaaS before any agents start reporting.	Critical
Follow the 4-wave deployment order	**Wave 1:** Management zones, auto-tagging rules, host groups. **Wave 2:** Service detection, request attributes, deep monitoring. **Wave 3:** Alerting profiles, anomaly detection, SLOs. **Wave 4:** Dashboards, remaining configs.	Critical
Use `oneagentctl --set-server` to reconfigure (not reinstall)	Run: `oneagentctl --set-server="https://{tenant}.live.dynatrace.com:443/communication"` then `--set-tenant-token="{token}"` then `systemctl restart oneagent`. Preserves host identity and custom metadata.	Critical
Restart application processes after migration	Full-stack monitoring requires process restarts—Java, .NET, Node.js, PHP all inject instrumentation at process startup. Without restart: host metrics flow but **no distributed tracing, no service detection, no code-level visibility**.	Critical
Migrate non-production first, then production	Validate data flow on non-prod hosts before touching production. Apply lessons learned to each subsequent wave.	Critical
Download deploy result CSVs for audit	After each wave, download the deployment result CSV from SaaS Upgrade Assistant. Archive these for compliance and troubleshooting.	Recommended
Use SaaS Upgrade Assistant selective import	Deploy configuration types incrementally using Smart Selective Import. Smart dependency management auto-adds required configs.	Recommended

Practice	Recommended Setting/Value	Priority
Preview all changes before deploying	Use the SaaS Upgrade Assistant **Preview Changes** feature to verify edits. The app retains original values for rollback reference.	Recommended
Manually recreate Credentials Vault entries	Cannot be exported. Re-enter all credentials (cloud platform integrations, extension credentials) directly in SaaS.	Critical
Generate new API tokens in SaaS	Managed tokens do not work in SaaS. Create new tokens with **minimal required scopes** for each use case.	Critical
Use Ansible/automation for bulk reconfiguration	For large environments, use configuration management (Ansible, Puppet, Chef) to execute `oneagentctl` commands and rolling application restarts across all hosts.	Recommended

6. Step 6: Integrate — Reconnect Integrations

Practice	Recommended Setting/Value	Priority
Recreate webhooks with SaaS URLs	Endpoint URLs change for SaaS. Create new Slack, Teams, PagerDuty, ServiceNow, and custom webhook integrations with SaaS URLs.	Critical
Update all API base URLs	Change from `{managed-url}/e/{env-id}` to `{tenant}.live.dynatrace.com`. Update tokens in all CI/CD pipelines and automation scripts.	Critical
Test alert delivery end-to-end	Trigger a test alert and confirm delivery through every notification channel (email, Slack, Teams, PagerDuty, ServiceNow).	Critical
Recreate synthetic private locations	Deploy new Synthetic ActiveGates in SaaS. Private locations are infrastructure-specific and do not transfer.	Recommended
Update CI/CD pipelines	Replace Managed API endpoints and tokens in all deployment pipelines, quality gates, and automated testing integrations.	Critical
Deploy dedicated AGs for Extensions 2.0	Extensions 2.0 require **host-based** ActiveGate (not K8s-based). Deploy a separate AG group for extension execution.	Recommended
Remove hardcoded entity IDs from dashboards	Before importing dashboards, replace hardcoded entity IDs with entity selectors. Update management zone IDs and dashboard owners.	Recommended

7. Step 7: Expand — Adopt New SaaS Capabilities

Practice	Recommended Setting/Value	Priority
Adopt Grail for unified querying	Replace ad-hoc USQL queries with Notebook-based DQL analysis. Grail provides a unified data lakehouse for logs, metrics, traces, events, and entities.	Recommended
Configure OpenPipeline for log masking	Configure log processing rules in OpenPipeline to redact PII (SSNs, credit card numbers, email addresses) before storage.	Recommended
Enable PII masking at ingest	Settings > Request attributes: enable masking on all attributes capturing user input, account numbers, or personal data. Configure session replay masking for RUM.	Critical
Implement Workflows for automation	Replace Managed problem notifications with SaaS Workflow triggers + HTTP Request actions. Recreate custom webhook logic as workflow steps.	Recommended
Right-size data retention for cost optimization	Configure Grail bucket retention by data type. Logs: set retention based on compliance needs. Metrics: leverage the 5-year aggregated retention. Use `bucket:` targeting in queries to limit scan scope.	Recommended
Explore Davis Copilot for AI-assisted analysis	Enable Davis Copilot for natural language querying and root cause analysis.	Optional

8. Step 8: Enable — User Enablement and Communication

Practice	Recommended Setting/Value	Priority
Communicate migration to all users	Send advance notice with migration timeline, expected impacts, new login URLs, and support channels. Communicate before, during, and after each migration wave.	Critical
Deliver persona-based training	Tailor training by role: SREs get DQL and alerting, developers get distributed tracing and code-level visibility, managers get dashboards and reporting.	Recommended
Update all documentation	Replace Managed URLs, procedures, and screenshots with SaaS equivalents in runbooks, architecture diagrams, onboarding guides, and disaster recovery plans.	Critical
Establish support channels	Create dedicated Slack/Teams channels for migration questions. Designate migration champions in each application team.	Recommended
Map Managed roles to SaaS IAM policies	Cluster admin > Account admin. Environment admin > Environment admin. Monitor user > Viewer + specific policies. Custom roles > Custom IAM policies.	Critical

Practice	Recommended Setting/Value	Priority
Replace LDAP with SAML	SaaS does not support direct LDAP authentication. Migrate to SAML 2.0 SSO through your IdP.	Critical
Filter SAML group claims to Dynatrace groups only	Azure Entra limit: **150 groups** per user in SAML claim. Filter to Dynatrace-related groups to stay under the limit.	Recommended

9. Step 9: Optimize — Validate, Optimize, and Decommission

Practice	Recommended Setting/Value	Priority
Allow 7–14 days for Davis AI baselines	Response time and error rate baselines: **2–7 days**. Resource usage and traffic patterns: **7–14 days**. Expect higher alert volume during this period—do not disable alerting.	Critical
Tune alert thresholds after baseline period	After 2 weeks: raise thresholds on noisy alerts, add time-based conditions, lower thresholds on missing alerts.	Recommended
Compare entity counts against inventory	Run `fetch dt.entity.host | summarize count()` (and service, application, process_group, synthetic_test). Counts must match the planning-phase inventory.	Critical
Verify metrics flow with no gaps >15 minutes	Run `timeseries avg(dt.host.cpu.usage), from:-1h, by: {dt.entity.host}` and check for nulls. Any host returning null has a data gap.	Critical
Validate log ingestion is continuous	Run `fetch logs, from:-1h | summarize count(), by: {bin(timestamp, 5m)}` and confirm no 5-minute buckets with zero count.	Critical
Confirm distributed traces are flowing	Run `fetch spans, from:-1h | summarize count(), by: {bin(timestamp, 5m)}` . Zero spans after agent migration means application processes need restart.	Critical
Get stakeholder sign-off	Obtain written approval from: Platform Team Lead, Security Team, Application Teams, Executive Sponsor.	Critical
Keep Managed running 2–4 weeks post-migration	Maintain access for historical reference and comparison. Decommission only after full validation period.	Recommended
Archive migration artifacts before decommission	Save SaaS Upgrade Assistant deploy result CSVs, entity inventory spreadsheets, and configuration mapping documents before decommissioning Managed.	Recommended
Validate all dashboard tiles show data	Open every migrated dashboard. Tiles with "No data" indicate missing entity mappings or incorrect management zone IDs.	Recommended
Set API token expiration dates	Set expiration on all tokens. Store in secrets management (HashiCorp Vault, AWS Secrets Manager). Rotate regularly.	Recommended

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.